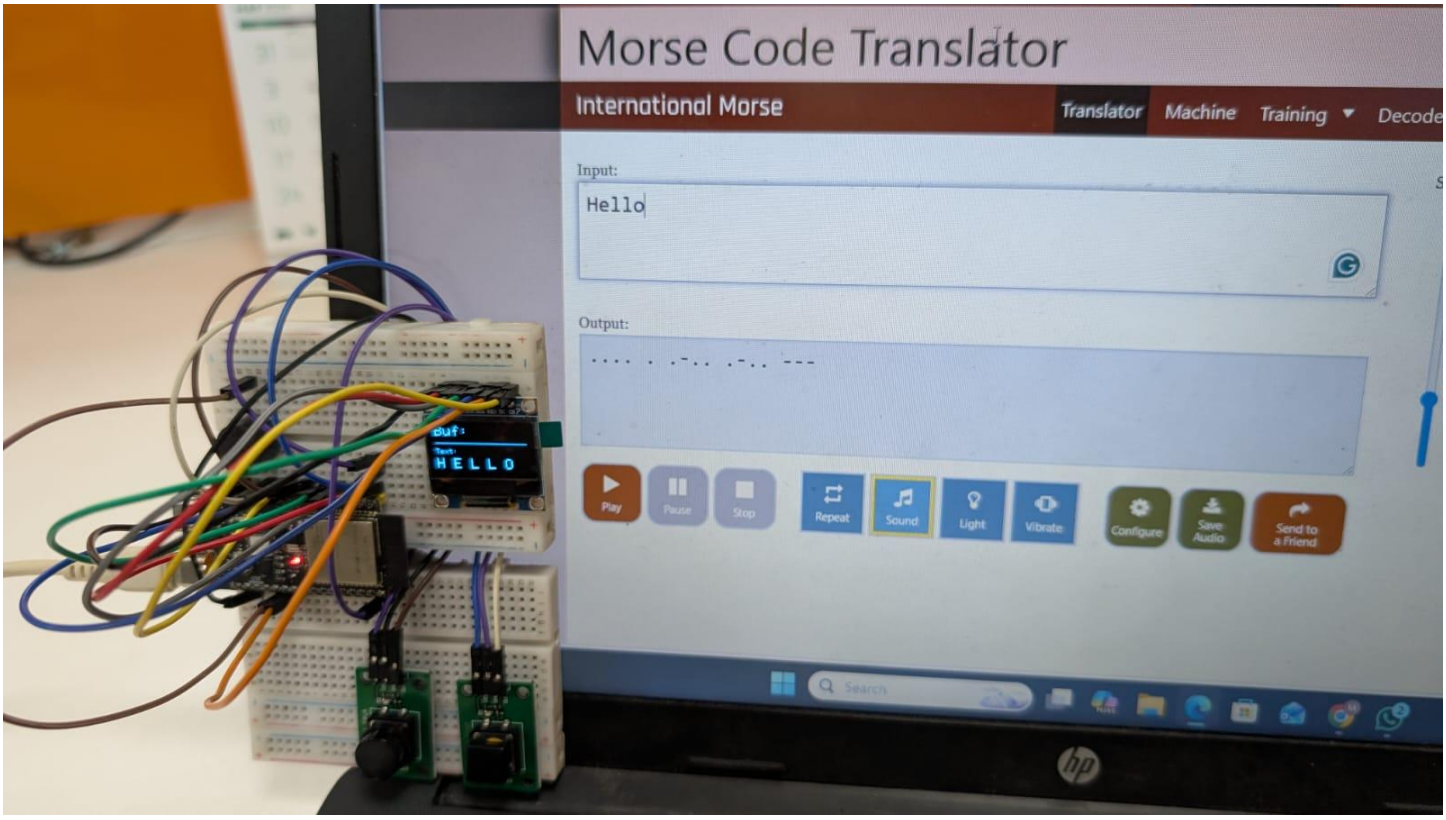
Morse Code Decoder Playbook

I. Challenge

Build a non-blocking embedded communication decoder that translates real-time manual dot/dash inputs into decoded alphanumeric text. The system must process debounced digital switch state changes, manage asynchronous timing windows (millis()) for dynamic character and word boundary isolation, provide instantaneous tactile/audio feedback, and render active output buffers on an SPI OLED display with zero processor stalling.



II. Guide

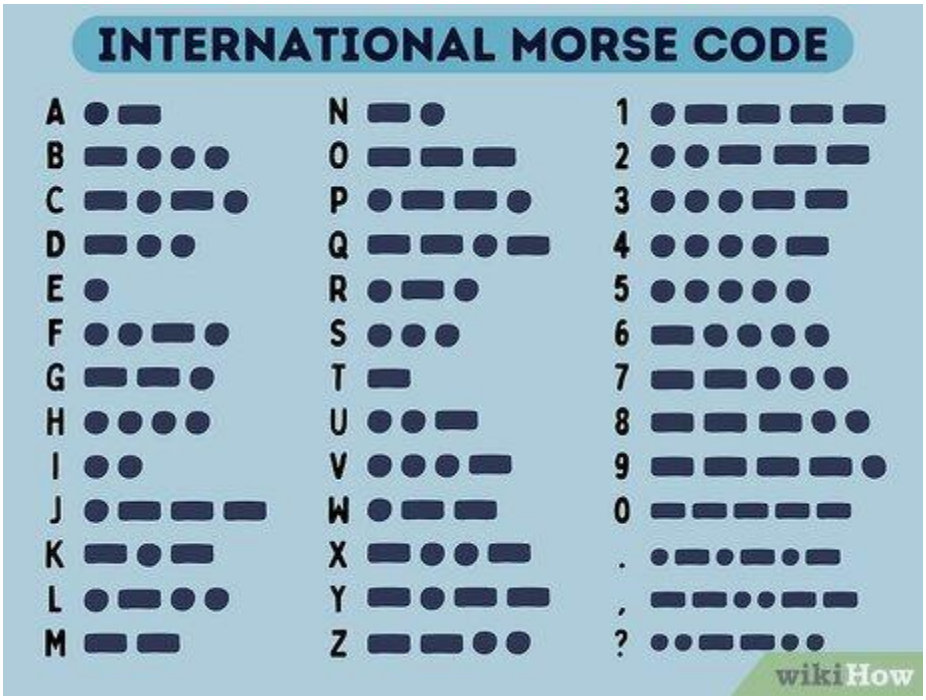
A. Concept

What is Morse Code?

Morse Code is a character encoding scheme designed in the 1830s for long-distance telecommunication via electric telegraph lines. It translates standard letters, numbers, and punctuation into rhythmic sequences of two discrete signal durations:

- **Dot (·):** The short signal duration, serving as the fundamental unit of time measurement (1 unit).
- **Dash (–):** The long signal duration, equal in length to exactly three dots (3 units).

In traditional telegraphy, timing intervals between signals define words and letters: spaces between dots/dashes within a single character equal **1 unit**, character gaps equal **3 units**, and word spaces equal **7 units**. In this embedded project, software timers (LETTER_TIMEOUT and WORD_TIMEOUT) automatically group dot/dash inputs and convert them into standard ASCII characters.



i. Underlying Engineering Principles & Reference Links

1. **Asynchronous Non-Blocking Event Loops (millis() Timing):** Using rigid delay() functions freezes microcontrollers and misses subsequent switch presses. Polling timestamps via millis() allows the system to continuously check inputs while evaluating background timers for letter timeouts (1000 ms) and word spaces (3000 ms).
    - *Learn More:* Arduino Documentation – Using millis() for Non-Blocking Timing
2. **State-Change Edge Detection:** Measuring continuous button levels triggers repeated inputs every millisecond a switch remains depressed. Edge detection tracks transitioning states (HIGH → LOW), capturing exactly one discrete event per physical button press.
    - *Learn More:* Arduino Documentation – State Change Detection (Edge Detection)
3. **Contact Switch Debouncing:** Mechanical switch contacts physically bounce upon actuation, causing microsecond-level noise spikes that register as multiple false presses. Combining hardware pull-up topologies with software debounce intervals stabilizes the logic level before registering an event.
    - *Learn More:* Digi-Key – Guide to Switch Debouncing Principles
4. **Binary Tree & Dictionary Parsing:** Morse code maps variable-length symbol sequences (· and –) directly to fixed alphanumeric ASCII characters. Looking up encoded buffer strings against a dictionary array converts time-domain signals into human-readable text.
    - *Learn More:* GeeksforGeeks – Binary Search Trees and Lookup Structures
5. **High-Speed Bus Communication (Hardware SPI):** Driving graphic displays via bit-banging consumes substantial processor cycles. Utilizing dedicated hardware Serial Peripheral Interface (SPI) buses offloads frame updates to high-speed shift registers.
    - *Learn More:* SparkFun – Serial Peripheral Interface (SPI) Architecture

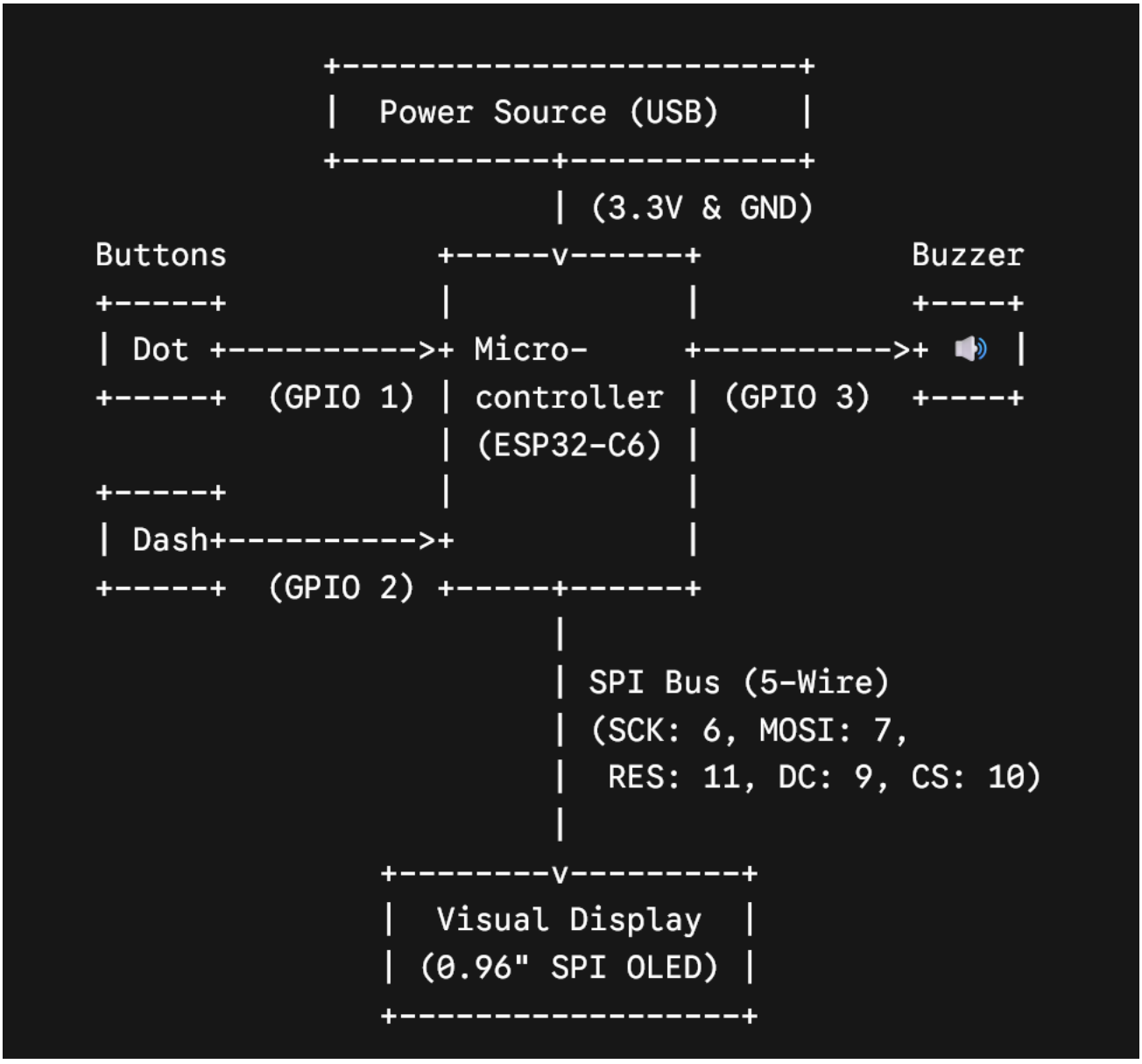
ii. Real-World Example

Aviation emergency radio beacons, naval signal lamps, telegraphy decoders, and accessibility switch interfaces (such as assistive sip-and-puff communication pads for motor-impaired individuals). These platforms translate discrete pulse duration patterns into digital text streams.

B. Components

- **Microcontroller:** WeAct Studio ESP32-C6 Dev Board
- **Display Interface:** 0.96-inch SPI OLED Display Module (SSD1306 driver)
- **Audio Transducer:** Active Buzzer Module (Active HIGH)
- **User Input:** 2× Push Button Switches (Active LOW with internal pull-up)
- **Interconnect Base:** Solderless breadboard and jumper wires

Block Diagram



C. Connections

The system operates entirely on 3.3V logic. Input push buttons utilize internal MCU pull-up resistors (INPUT_PULLUP), pulling the pin to GND (0V) upon activation.

Peripheral Component	Component Pin	ESP32-C6 Pin	Function / Designation
SPI OLED	GND	GND	Common Ground Plane
SPI OLED	VCC	3.3V	Logic Power Supply
SPI OLED	SCLK (D0)	GPIO 6	Hardware SPI Clock Line
SPI OLED	MOSI (D1)	GPIO 7	Hardware SPI MOSI Data Line
SPI OLED	RES (RST)	GPIO 11	Hardware Reset Line
SPI OLED	DC (A0)	GPIO 9	Data/Command Mode Select
SPI OLED	CS	GPIO 10	Active-Low Peripheral Chip Select
Dot Input Button	OUT / Signal	GPIO 1	Debounced Dot State-Change Input
Dash Input Button	OUT / Signal	GPIO 2	Debounced Dash State-Change Input
Active Buzzer	I/O / Signal	GPIO 3	Audio Actuation Output (Active HIGH)

D. Code

```
C++
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// --- Display Configuration ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define OLED_MOSI   7
#define OLED_CLK    6
#define OLED_DC     9
#define OLED_CS    10
#define OLED_RESET 11

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, OLED_MOSI, OLED_CLK, OLED_DC, OLED_RESET, OLED_CS);

// --- GPIO Definitions ---
const int DOT_BUTTON  = 1;
const int DASH_BUTTON = 2;
```

```
const int BUZZER      = 3;

// --- Timing & State Variables ---
const unsigned long LETTER_TIMEOUT = 1000;
const unsigned long WORD_TIMEOUT   = 3000;
const int AUDIO_FEEDBACK_MS = 80;

String morseBuffer = "";
String decodedText = "";
unsigned long lastInputTime = 0;

bool letterPending = false;
bool wordPending = false;

int lastDotState = HIGH;
int lastDashState = HIGH;

// --- Translation Logic ---
char translateMorse(String morse) {
  if (morse == ".-")   return 'A'; if (morse == "-...") return 'B';
  if (morse == "-.-.") return 'C'; if (morse == "-..")  return 'D';
  if (morse == ".")    return 'E'; if (morse == "..-.") return 'F';
  if (morse == "--.")  return 'G'; if (morse == "....") return 'H';
  if (morse == "..")   return 'I'; if (morse == ".---") return 'J';
  if (morse == "-.-")  return 'K'; if (morse == ".-..") return 'L';
  if (morse == "--")   return 'M'; if (morse == "-.")   return 'N';
  if (morse == "---")  return 'O'; if (morse == ".--.") return 'P';
  if (morse == "--.-") return 'Q'; if (morse == ".-.")  return 'R';
  if (morse == "...")  return 'S'; if (morse == "-")    return 'T';
  if (morse == "..-")  return 'U'; if (morse == "...-") return 'V';
  if (morse == ".--")  return 'W'; if (morse == "-..-") return 'X';
  if (morse == "-.--") return 'Y'; if (morse == "--..") return 'Z';

  if (morse == "-----") return '0'; if (morse == ".----") return '1';
  if (morse == "..---") return '2'; if (morse == "...--") return '3';
  if (morse == "....-") return '4'; if (morse == ".....") return '5';
  if (morse == "-....") return '6'; if (morse == "--...") return '7';
  if (morse == "---..") return '8'; if (morse == "----.") return '9';

  return '?';
}

// --- Subroutines ---
void updateDisplay() {
  display.clearDisplay();

  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.print("Buf:");
  display.print(morseBuffer);

  display.drawFastHLine(0, 24, 128, SSD1306_WHITE);

  display.setTextSize(1);
  display.setCursor(0, 32);
  display.print("Text:");
  display.setCursor(0, 45);
  display.setTextSize(2);

  // Truncate output to prevent visual overflow
  if (decodedText.length() > 10) {
    display.print(decodedText.substring(decodedText.length() - 10));
  } else {
    display.print(decodedText);
  }

  display.display();
}

void triggerBuzzer() {
  digitalWrite(BUZZER, HIGH);
```

```
  delay(AUDIO_FEEDBACK_MS);
  digitalWrite(BUZZER, LOW);
}

// --- Initialization ---
void setup() {
  Serial.begin(115200);

  pinMode(DOT_BUTTON, INPUT_PULLUP);
  pinMode(DASH_BUTTON, INPUT_PULLUP);

  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);

  if(!display.begin(SSD1306_SWITCHCAPVCC)) {
    for(;;); // System halt on peripheral failure
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(10, 20);
  display.print("System Ready");
  display.display();
  delay(1500);

  updateDisplay();
  lastInputTime = millis();
}

// --- Main Execution Loop ---
void loop() {
  unsigned long currentTime = millis();

  int currentDotState = digitalRead(DOT_BUTTON);
  int currentDashState = digitalRead(DASH_BUTTON);

  // Event Trigger: Dot Input
  if (currentDotState == LOW && lastDotState == HIGH) {
    morseBuffer += ".";
    triggerBuzzer();
    lastInputTime = currentTime;
    letterPending = true;
    wordPending = true;
    updateDisplay();
    delay(40);
  }
  lastDotState = currentDotState;

  // Event Trigger: Dash Input
  if (currentDashState == LOW && lastDashState == HIGH) {
    morseBuffer += "-";
    triggerBuzzer();
    lastInputTime = currentTime;
    letterPending = true;
    wordPending = true;
    updateDisplay();
    delay(40);
  }
  lastDashState = currentDashState;

  // Async Execution: Letter Translation
  if (letterPending && (currentTime - lastInputTime >= LETTER_TIMEOUT)) {
    char newChar = translateMorse(morseBuffer);
    decodedText += newChar;

    // Diagnostic Serial Telemetry
    Serial.print("Decoded_Char:"); Serial.print(newChar); Serial.print(",");
    Serial.print("Full_Text:"); Serial.println(decodedText);

    morseBuffer = "";
    letterPending = false;
```

```
    updateDisplay();
  }

  // Async Execution: Word Spacing
  if (wordPending && !letterPending && (currentTime - lastInputTime >= WORD_TIMEOUT)) {
    decodedText += " ";
    wordPending = false;

    Serial.print("Event:"); Serial.println("WORD_SPACE_ADDED");
    updateDisplay();
  }

}
```

Student Tip: Open the **Serial Monitor** (Ctrl+Shift+M) at **115200 baud** to trace decoded characters and watch letter timeouts trigger in real time.

You can also open the **Serial Plotter** (Ctrl+Shift+L) to inspect timing delays visually. Watching telemetry events on the plotter will demonstrate how LETTER_TIMEOUT (1000 ms) and WORD_TIMEOUT (3000 ms) reset dynamic timers after every keystroke.

III. Play & Share

- **Single-Key Auto-Length Timing (Single iambic keyer):** Transform the hardware setup from a dual-button configuration into a single key system. Write an algorithm that measures press duration using millis()—short taps (< 200 ms) record a **Dot**, while holding the key longer (> 200 ms) records a **Dash**.
- **Implement Adaptive WPM Speed Tuning:** Add a 10 kΩ potentiometer to an analog input pin. Scale its 12-bit ADC reading ($0 - 4095$) to dynamically adjust LETTER_TIMEOUT between 400 ms (expert fast operator) and 2000 ms (beginner slow operator).
- **Morse Code Training Game:** Reverse the application! Program the ESP32-C6 to display a random ASCII letter on the OLED, play its audio Morse signature through the buzzer, and time how quickly the student can correctly re-enter the matching Dot/Dash combination.

Use this section in case of parts variation

Hello! This section is designed to assist in the case of components varience. In simpler terms, did you get a component that has the same functions but the connections are different?

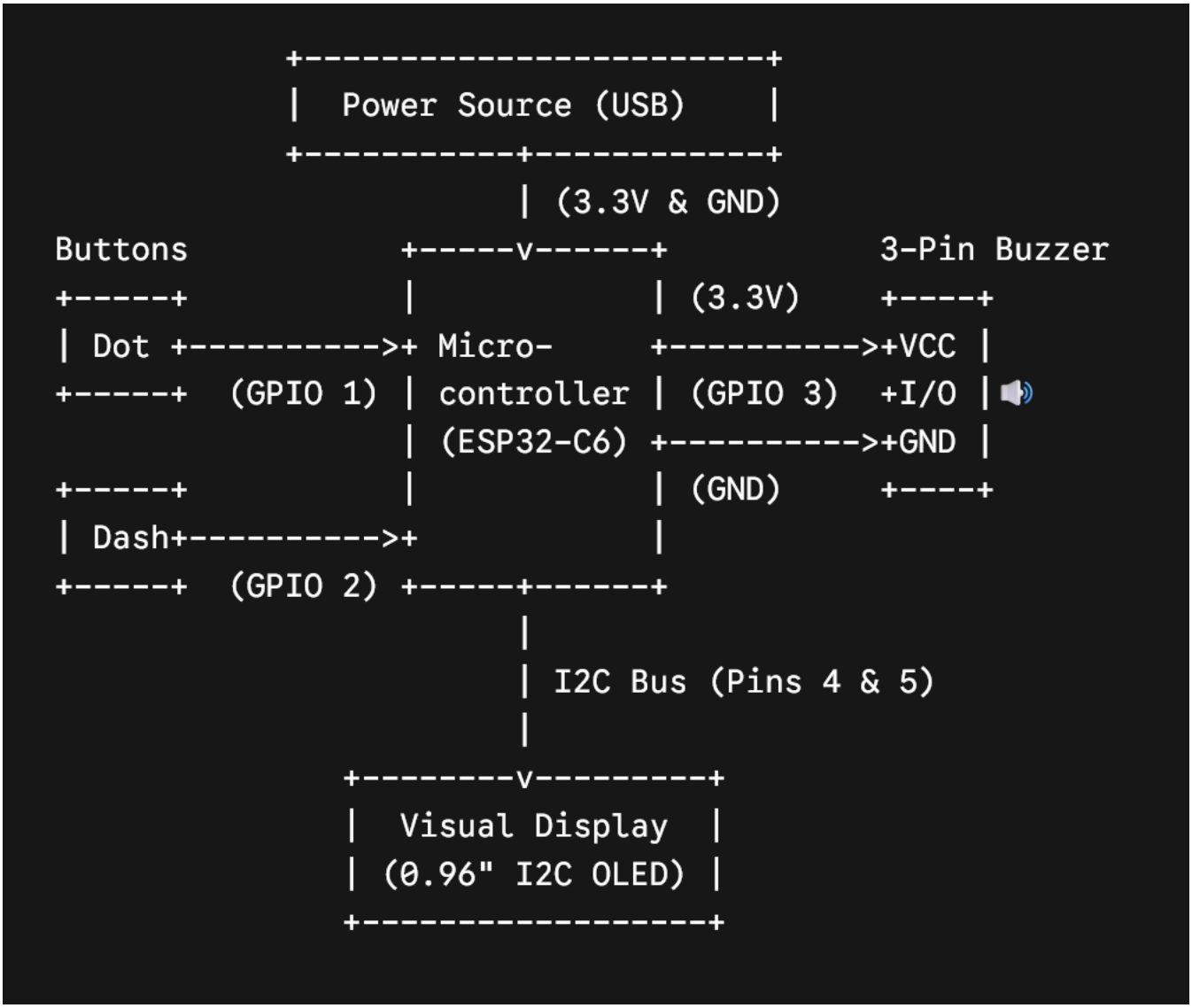
Some examples include:

- Your OLED Display has 4 pins instead of 7 pins? It turns out you got an I2C Version!
- Your buzzer module is either 2 pins or 3 pins

For our Morse Code Decoder, we are now working with:

- 4-pin I2C OLED
- 3-pin buzzer module

Here is the new block diagram:



New Connections Table

You will now route the power pins of the buzzer to the shared power rails on your breadboard alongside the OLED screen.

Component	ESP32-C6 Pin	Description
Dot Button	GPIO 1	The button to tap for a short "Dot".
Dash Button	GPIO 2	The button to tap for a long "Dash".
Buzzer VCC	3.3V or 5V	Provides main power to the buzzer module.
Buzzer GND	GND	Common ground connection (Minus).
Buzzer I/O	GPIO 3	Triggers the beep sound when the code sends a HIGH signal.
OLED GND	GND	Common ground connection (Minus).
OLED VCC	3.3V	Logic power to turn the screen on.
OLED SDA	GPIO 4	The I2C Data line for sending text to the screen.
OLED SCL	GPIO 5	The I2C Clock line to time the screen data.

Code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// --- Shared I2C Pin Configuration ---
#define I2C_SDA 4
#define I2C_SCL 5

// --- Display Configuration ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1 // Not used for I2C screens

// Initialize the I2C OLED
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// --- GPIO Definitions ---
// BUTTON WIRING: Connect one pin of the button to the GPIO, and the other directly to GND.
const int DOT_BUTTON  = 1;
const int DASH_BUTTON = 2;
```

```
21.
22. // BUZZER WIRING: Connect I/O pin to GPIO 3, VCC pin to 3.3V or 5V, and GND pin to GND.
23. const int BUZZER      = 3;
24.
25. // --- Timing Variables (Stopwatches) ---
26. const unsigned long LETTER_TIMEOUT = 1000; // 1 second pause = New Letter
27. const unsigned long WORD_TIMEOUT   = 3000; // 3 second pause = New Word
28. const int AUDIO_FEEDBACK_MS = 80;          // How long the beep lasts
29.
30. String morseBuffer = "";
31. String decodedText = "";
32. unsigned long lastInputTime = 0;
33.
34. bool letterPending = false;
35. bool wordPending = false;
36.
37. int lastDotState = HIGH;
38. int lastDashState = HIGH;
39.
40. // --- Translation Dictionary ---
41. char translateMorse(String morse) {
42.   if (morse == ".-")   return 'A'; if (morse == "-...") return 'B';
43.   if (morse == "-.-.") return 'C'; if (morse == "-..")  return 'D';
44.   if (morse == ".")    return 'E'; if (morse == "..-.") return 'F';
45.   if (morse == "--.")  return 'G'; if (morse == "....") return 'H';
46.   if (morse == "..")   return 'I'; if (morse == ".---") return 'J';
47.   if (morse == "-.-")  return 'K'; if (morse == ".-..") return 'L';
48.   if (morse == "--")   return 'M'; if (morse == "-.")   return 'N';
49.   if (morse == "---")  return 'O'; if (morse == ".--.") return 'P';
50.   if (morse == "--.-") return 'Q'; if (morse == ".-.")  return 'R';
51.   if (morse == "...")  return 'S'; if (morse == "-")    return 'T';
52.   if (morse == "..-")  return 'U'; if (morse == "...-") return 'V';
53.   if (morse == ".--")  return 'W'; if (morse == "-..-") return 'X';
54.   if (morse == "-.--") return 'Y'; if (morse == "--..") return 'Z';
55.
56.   if (morse == "-----") return '0'; if (morse == ".----") return '1';
57.   if (morse == "..---") return '2'; if (morse == "...--") return '3';
58.   if (morse == "....-") return '4'; if (morse == ".....") return '5';
59.   if (morse == "-....") return '6'; if (morse == "--...") return '7';
60.   if (morse == "---..") return '8'; if (morse == "----.") return '9';
61.
62.   return '?'; // If the pattern doesn't exist
63. }
64.
65. // --- Screen Drawing Function ---
66. void updateDisplay() {
67.   display.clearDisplay();
68.
69.   display.setTextSize(2);
70.   display.setTextColor(SSD1306_WHITE);
71.   display.setCursor(0, 0);
72.   display.print("Buf:");
73.   display.print(morseBuffer); // Show dots and dashes
74.
75.   display.drawFastHLine(0, 24, 128, SSD1306_WHITE);
76.
77.   display.setTextSize(1);
78.   display.setCursor(0, 32);
79.   display.print("Text:");
80.   display.setCursor(0, 45);
81.   display.setTextSize(2);
82.
83.   // Cut off long text so it fits on the screen
84.   if (decodedText.length() > 10) {
85.     display.print(decodedText.substring(decodedText.length() - 10));
86.   } else {
87.     display.print(decodedText);
88.   }
89.
90.   display.display();
91. }
92.
```

```
93. void triggerBuzzer() {
94.   digitalWrite(BUZZER, HIGH);
95.   delay(AUDIO_FEEDBACK_MS);
96.   digitalWrite(BUZZER, LOW);
97. }
98.
99. void setup() {
100.   Serial.begin(115200);
101.
102.   // Setup I2C Pins for the OLED
103.   Wire.begin(I2C_SDA, I2C_SCL);
104.
105.   // Setup the buttons with internal pull-up resistors
106.   pinMode(DOT_BUTTON, INPUT_PULLUP);
107.   pinMode(DASH_BUTTON, INPUT_PULLUP);
108.
109.   // Setup the buzzer output
110.   pinMode(BUZZER, OUTPUT);
111.   digitalWrite(BUZZER, LOW);
112.
113.   // Start the OLED screen (0x3C is the standard I2C address)
114.   if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
115.     Serial.println("OLED not found! Check wiring.");
116.     for(;;); // Freeze if screen fails
117.   }
118.
119.   display.clearDisplay();
120.   display.setTextSize(1);
121.   display.setTextColor(SSD1306_WHITE);
122.   display.setCursor(10, 20);
123.   display.print("System Ready");
124.   display.display();
125.   delay(1500);
126.
127.   updateDisplay();
128.   lastInputTime = millis();
129. }
130.
131. void loop() {
132.   unsigned long currentTime = millis();
133.
134.   // Read the current state of both buttons
135.   int currentDotState = digitalRead(DOT_BUTTON);
136.   int currentDashState = digitalRead(DASH_BUTTON);
137.
138.   // 1. Did the user press the DOT button? (State changed from HIGH to LOW)
139.   if (currentDotState == LOW && lastDotState == HIGH) {
140.     morseBuffer += ".";
141.     triggerBuzzer();
142.     lastInputTime = currentTime; // Reset stopwatch
143.     letterPending = true;
144.     wordPending = true;
145.     updateDisplay();
146.     delay(40); // Small software delay to ignore bouncy metal contacts
147.   }
148.   lastDotState = currentDotState;
149.
150.   // 2. Did the user press the DASH button? (State changed from HIGH to LOW)
151.   if (currentDashState == LOW && lastDashState == HIGH) {
152.     morseBuffer += "-";
153.     triggerBuzzer();
154.     lastInputTime = currentTime; // Reset stopwatch
155.     letterPending = true;
156.     wordPending = true;
157.     updateDisplay();
158.     delay(40); // Small software delay to ignore bouncy metal contacts
159.   }
160.   lastDashState = currentDashState;
161.
162.   // 3. Did 1 second pass? Translate the letter!
163.   if (letterPending && (currentTime - lastInputTime >= LETTER_TIMEOUT)) {
164.     char newChar = translateMorse(morseBuffer);
```

```
    decodedText += newChar;

    Serial.print("Decoded_Char:"); Serial.print(newChar); Serial.print(",");
    Serial.print("Full_Text:"); Serial.println(decodedText);

    morseBuffer = "";
    letterPending = false;
    updateDisplay();
  }

  // 4. Did 3 seconds pass? Add a space!
  if (wordPending && !letterPending && (currentTime - lastInputTime >= WORD_TIMEOUT)) {
    decodedText += " ";
    wordPending = false;

    Serial.print("Event:"); Serial.println("WORD_SPACE_ADDED");
    updateDisplay();
  }
}

```